

EDMONDS – KARP VISUALIZER

BFS-Based Maximum Flow Algorithm Visualization

Team Members:

- P Yasasri – 2520030373
- M Sri Nithya – 2520030447

Team Number: 23

Section: 03

A G E N D A

Project Contents

- | | |
|-----------------------------|------------------|
| 01. Abstract | 07. Future Scope |
| 02. Problem Statement | 08. Conclusion |
| 03. Project Objective | |
| 04. DSA Concepts Used | |
| 05. Tools and technologies | |
| 06. System Architecture | |
| 07. Frontend Implementation | |

ABSTRACT

- Edmonds–Karp is a maximum flow algorithm based on BFS.
- It repeatedly finds an augmenting path from source to destination.
- The flow is increased by the minimum available capacity on that path.
- After each iteration, the residual graph is updated.
- Our system visualizes these steps and displays the final maximum flow.

PROBLEM STATEMENT

- Finding maximum flow requires repeatedly selecting suitable augmenting paths.
- In Edmonds–Karp, BFS selects the shortest available augmenting path.
- After sending flow, the residual capacities change.
- Tracking these changes through multiple iterations can be difficult manually.
- Our project aims to visualize each BFS path and residual graph update step-by-step.

PROJECT OBJECTIVE

- Our project aims to develop an interactive web-based visualizer for the Edmonds–Karp maximum flow algorithm.

Objectives:

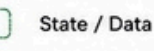
- Create and modify a directed flow network.
- Define edge capacities.
- Select source and destination nodes.
- Find augmenting paths using BFS.
- Visualize residual graph updates.
- Display bottleneck capacity and flow added at each iteration.
- Calculate and display the final maximum flow.

DSA CONCEPTS USED

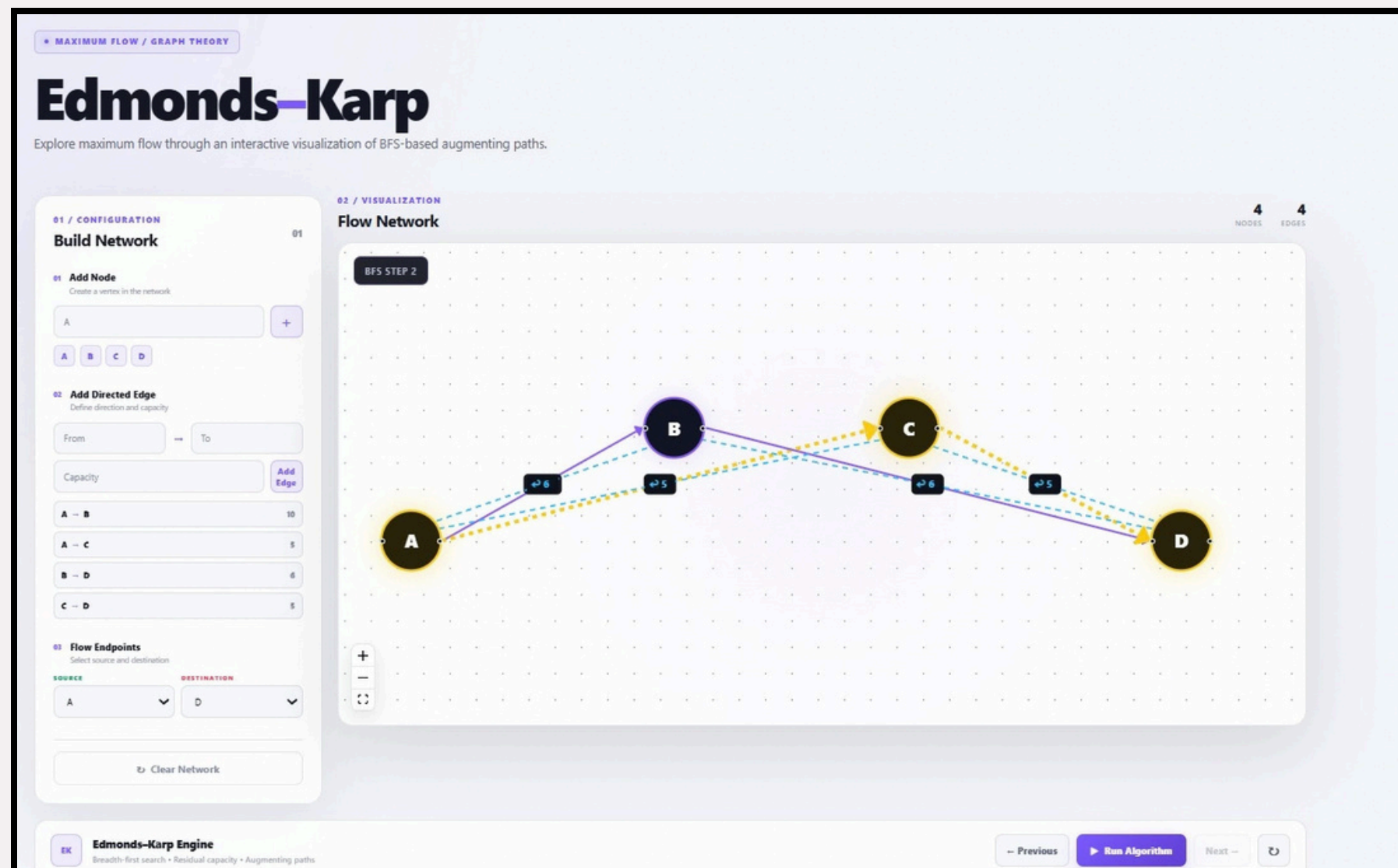
- Graph
 - Directed Graph
 - Adjacency representation
- Queue
 - Used in BFS traversal
- BFS
 - Finds the shortest augmenting path
- Residual Graph
 - Stores remaining capacities after flow updates
- Edmonds–Karp
 - Repeatedly applies BFS to find augmenting paths
 - Updates residual capacities
 - Calculates maximum flow

TOOLS/TECHNOLOGIES USED

- Frontend
 - React
 - JavaScript
 - HTML
 - CSS
- Development
 - Vite
- Graph Visualization
 - React Flow
- Algorithm
 - Edmonds–Karp
 - Breadth-First Search
 - Residual Graph



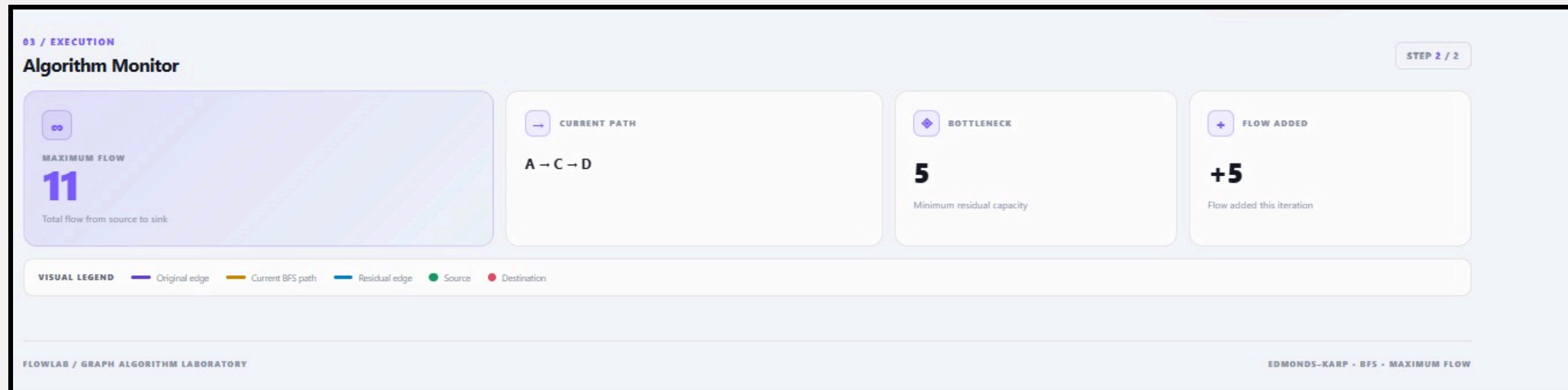
FRONTEND IMPLEMENTATION



Implemented Features

- Add nodes dynamically.
- Add directed edges with capacities.
- Select source and destination.
- Interactive graph visualization.
- Run Edmonds–Karp algorithm.
- Navigate through algorithm steps.
- Reset and clear the network.

ALGORITHM VISUALIZATION



The system displays:

- Current BFS augmenting path
- Bottleneck capacity
- Flow added in the current iteration
- Current maximum flow
- Current algorithm step
- Residual graph changes
- Visual legend for graph states

FUTURE SCOPE

- Add support for other maximum-flow algorithms such as Ford–Fulkerson and compare their execution.
- Add more interactive controls and detailed step-by-step explanations of the algorithm.
- Improve graph visualization and provide more learning-oriented information for students.

CONCLUSION

- Successfully developed an interactive Edmonds–Karp Maximum Flow Visualizer.
- Provides an intuitive way to create graphs, add capacities, and select source and destination.
- Visualizes BFS-based augmenting paths, bottleneck capacity, residual edges, and flow updates step-by-step.
- Helps users understand the working of the Edmonds–Karp algorithm beyond just the final answer.
- Combines algorithmic logic with an interactive React-based visualization for better learning and understanding.
- The project demonstrates how data structures, graph algorithms, and modern web technologies can be integrated into a practical educational tool.

THANK YOU